

개인 회고

EfficientDet 실습을 진행하면서 막혔던 부분과 해결 방법, 그리고 개인 회고를 정리했다.

진행하면서 막혔던 부분과 해결

막힌 부분	원인	해결
원본 노트북을 이 PC에서 그대로 실행할 수 없음	원본은 Colab용이라 <code>/content/...</code> 경로, <code>wget</code> , <code>mkdir -p</code> 가 들어 있다	이 PC에서 안 되는 곳만 고치고 코랩용 과 노트북용을 나란히 표시했다. 원본 코드 셀 57개 중 43개는 글자 하나까지 그대로다
<code>!ls -l</code> , <code>!pwd</code> 가 'ls' is not recognized... 에러	이 PC의 Jupyter는 ! 명령을 Windows 명령창(cmd)으로 실행해서 <code>ls</code> · <code>pwd</code> · <code>unzip</code> · <code>grep</code> 이 없다. 처음엔 Git Bash에서 시험해 된다고 잘못 판단했다	<code>os.getcwd()</code> , <code>os.listdir</code> , <code>zipfile</code> , <code>dir /b /od</code> 로 바꿨다
<code>soccer.mp4</code> 가 없음	원본 Colab 실행 기록에도 <code>False</code> 였다. 실제로 쓰인 파일은 <code>soccer.gif</code> 였다	<code>soccer.gif</code> 를 받아 <code>video_src</code> 를 바꿨다
yaml 메모대로 하면 데이터를 못 찾음	<code>project_name</code> 이 폴더 이름과 달랐다	<code>my_car_detect_proj</code> 로 맞췄다. 원본 Colab 로그도 이 이름의 폴더에 저장되어 있었다
<code>num_gpus: 0</code> 이면 CPU로 학습함	<code>train.py</code> 가 0이면 GPU를 끈다	<code>num_gpus: 1</code> 로 바꿨다
학습이 시작되지 않음 (UnicodeDecodeError: 'cp949')	yaml 파일 안에 한글·이모지 주석을 넣었더니 <code>train.py</code> 가 yaml을 cp949로 읽다가 실패했다. ! 명령은 실패해도 셀이 멈추지 않아 다음 셀에서야 드러났다	yaml은 파이썬으로 쓰고 파일 안에는 원본 영어 주석만 남겼다
<code>ReduceLROnPlateau(..., verbose=True)</code> 에러	최신 PyTorch에서 <code>verbose</code> 인자가 없어졌다	원본 안내대로 지웠다
학습이 에폭당 3분 30초 걸림	워커 12개를 Windows에서는 에폭마다 새로 띄운다. 그동안 GPU 사용률은 0%였다	<code>-n 0</code> 으로 에폭당 약 15초가 됐다
<code>!python train.py</code> 실패	이 PC에서 <code>python</code> 은 <code>pycocotools</code> 가 없는 시스템 파이썬을 가리킨다	<code>{sys.executable}</code> 로 커널의 파이썬을 썼다
PCB 사진 추론에서 'NoneType' object is not subscriptable	커널이 이미 레포 안에 있는 상태에서 <code>git clone</code> · <code>os.chdir</code> 를 다시 실행해 레포가 두 겹으로 받아졌고, 안쪽 사본에는 이미지가 없었다	안쪽 사본을 지우고 커널을 재시작해 처음부터 다시 실행했다. 흐름 1은 커널 재시작 직후에만 실행한다

실습을 통해 알게 된 것

이번 실습에서 가장 크게 남은 것은 탐지 모델 안에서 실제로 어떤 일이 일어나는지에 대한 이론이다. 먼저 특징맵 P3~P7의 숫자 N은 이미지를 2^N 배로 줄여서 본다는 뜻이고, 많이 줄인 P7은 칸 하나가 원본의 넓은 영역을 요약해서 보기 때문에 큰 물체를, 칸이 촘촘한 P3은 작은 물체를 맡는다는 것을 알았다. P3은 위치가 정확하지만 의미 정보가 약하고 P7은 그 반대라서, EfficientDet은 BiFPN으로 둘을 섞어 서로의 약점을 보완한다.

앵커의 역할도 이해했다. 앵커는 특징맵의 칸마다 미리 깔아 두는 기준 박스로, 모델은 박스를 처음부터 그리지 않고 앵커를 기준으로 얼마나 옮기고 늘릴지만 예측한다. 앵커는 학습과 예측 내내 바뀌지 않고, 칸마다 9개씩 깔려서 D0에서는 49,104개가 된다. 학습할 때는 앵커와 정답 박스의 IoU로 역할을 정하는데, 0.5 이상이면 그 물체를 배우고 0.4 미만이면 배경으로 배우며, 그 사이의 애매한 앵커는 학습에서 뺀다. 모델이 정답을 스스로 아는 것이 아니라, 사람이 만든 라벨을 손실 함수가 예측과 비교해 얼마나 틀렸는지 알려주는 구조라는 것도 이때 알게 됐다.

마지막으로 Focal Loss가 왜 필요한지 알게 됐다. 찾으려는 물체에 비해 배경 앵커가 압도적으로 많아서, 그냥 학습하면 배경의 손실이 쌓여 물체의 손실이 묻혀 버린다. Focal Loss는 이미 잘 맞힌 쉬운 배경의 손실을 크게 줄여서, 모델이 어려운 물체를 배우는 데 집중하게 한다.

어려웠던 것

특징맵에서 왜 P7이 큰 물체를, P3이 작은 물체를 맡는지 이해하기 어려웠다. 처음에는 이미지를 줄여서 보는데 왜 큰 물체를 담당하는지 납득이 되지 않았는데, 칸 하나가 원본의 얼마만큼을 요약해서 보는지로 생각하니 이해가 됐다. Colab용 원본 코드를 내 컴퓨터에서 돌리는 것도 어려웠다. 경로, 리눅스 명령, 파이썬 위치처럼 환경 차이 때문에 막히는 곳이 많았다.

좋았던 점과 아쉬웠던 점

이론적인 부분을 많이 알아간 것 같아서 기분이 좋았다. 반면 Colab 코드를 내 컴퓨터에 맞게 바꾸는 데 시간이 많이 들었고, 이론 내용을 한 번에 이해하지 못했다. 듣고 이해한 것 같아도 막상 배운 내용을 내 말로 설명하려고 하면 막혔다.

처음 기대와 실제

솔직히 처음에는 큰 기대가 없었다. 그런데 막상 해 보니 이론적인 내용이 흥미로워서 공부하는 것이 즐거웠다. 교수님 말씀대로 중학생도 이해할 수 있게 설명하는 훈련을 해야겠다고 느꼈고, 이번 실습에서 얻은 것도 새로운 지식과 함께 바로 이 마음가짐이다.

현장에 쓴다면

솔직히 지금은 YOLO와 크게 다르지 않아 보이고, 개인적으로는 YOLO가 더 좋아 보여서 현장에서는 YOLO를 쓸 것 같다. 다만 자원이 부족한 곳에서는 적은 연산량으로 높은 효율을 내는 EfficientDet이 더 좋을 것 같다.

앞으로 해 볼 것

앞으로 있을 중간고사에 대비해 배운 내용을 쉽게 설명하는 방법을 연습할 예정이다. 또 입력 이미지 크기를 바꿔(D1은 640, D2는 768) 학습해 보면서 앵커 개수가 어떻게 달라지는지 직접 구해 보고 싶다.

동료 피드백

받은 피드백은 없다.